Compte rendu de travaux pratiques

Programmation embarquée

Entrées et sorties GPIO du STM32F407 :
feu tricolore et bouton poussoir

Enseignant de TP

M. Atef Ftirich

Réalisé par

Bouabdallah Mohamed Yacine
Arrami Mohamed Ali
Ben Hmida Ahmed

Classe : LISI3

Groupe : A

Année universitaire : 2026/2027

Sommaire

1. Introduction et objectifs..... 2
2. Matériel et brochage..... 2
2.1 Matériel utilisé..... 2
2.2 Affectation des broches..... 3
3. Rappel : les registres GPIO utilisés..... 3
4. Partie 1 : rappel – clignotement d'une LED sur PA5..... 4
5. Partie 2 : feu tricolore..... 5
5.1 Programme..... 5
5.2 Explication du programme..... 5
5.3 Durée des temporisations..... 6
5.4 Résultats..... 7
6. Partie 3 : commande d'une LED par un bouton poussoir..... 8
6.1 Programme saisi..... 8
6.2 Explication du programme..... 9
6.3 Erreurs relevées dans le programme..... 9
6.4 Programme corrigé..... 10
6.5 Montage et comportement attendu..... 11
7. Points d'attention..... 11
8. Conclusion..... 12
Remerciements..... 12

1. Introduction et objectifs

Le premier TP de programmation embarquée nous a permis de préparer l'environnement Keil µVision et de commander une sortie du microcontrôleur STM32F407 en accédant directement à ses registres. Ce deuxième TP poursuit ce travail sur la carte STM32F4 Discovery (référence STM32F407G-DISC1), toujours sans générateur de code et uniquement avec Keil.

Objectifs du TP :

- vérifier le montage et la chaîne de programmation avec le clignotement d'une LED externe sur PA5 ;
- réaliser un feu tricolore à trois LED commandées par le registre ODR et une temporisation logicielle ;
- configurer des broches en entrée avec résistance de tirage interne (registre PUPDR) et lire leur état (registre IDR) ;
- commander une LED avec un bouton poussoir, en utilisant le registre BSRR ;
- analyser les programmes, détecter les erreurs et proposer les corrections nécessaires.

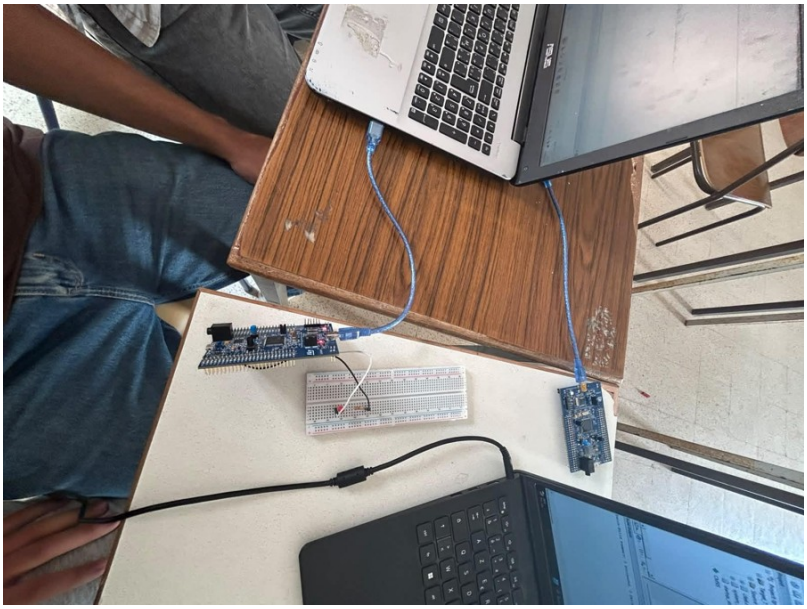


Figure 1 – *Poste de travail : carte reliée au PC par le câble USB du ST-LINK*

2. Matériel et brochage

2.1 Matériel utilisé

Composant	Rôle
Carte STM32F4 Discovery (STM32F407VG)	Microcontrôleur Cortex-M4 programmé en C, avec programmateur ST-LINK intégré.
Câble USB (mini-USB)	Alimentation de la carte et téléversement du programme par le ST-LINK.
Breadboard et fils	Réalisation des connexions sans soudure.
Un bouton poussoir	Entrée de commande de la partie 3.
LED rouge, jaune et verte + résistances 220 Ω	Sorties : LED de la partie 2 et feu tricolore de la partie 3.

Composant	Rôle
Keil µVision 5, pack STM32F4xx_DFP 2.17.1	Écriture, compilation et téléversement (CMSIS CORE + Device Startup).

2.2 Affectation des broches

Partie	Broche	Mode	Rôle
1	PA5	Sortie	LED externe qui clignote
2	PA5 / PA6 / PA7	Sortie	LED rouge / jaune / verte du feu tricolore
3	PA0	Entrée, pull-up	Bouton poussoir câblé sur la breadboard
3	PA1	Entrée, pull-up	Configurée dans le programme, mais aucun bouton n'y est relié
3	PD12	Sortie	LED commandée (aussi reliée à la LED verte LD4 de la carte)

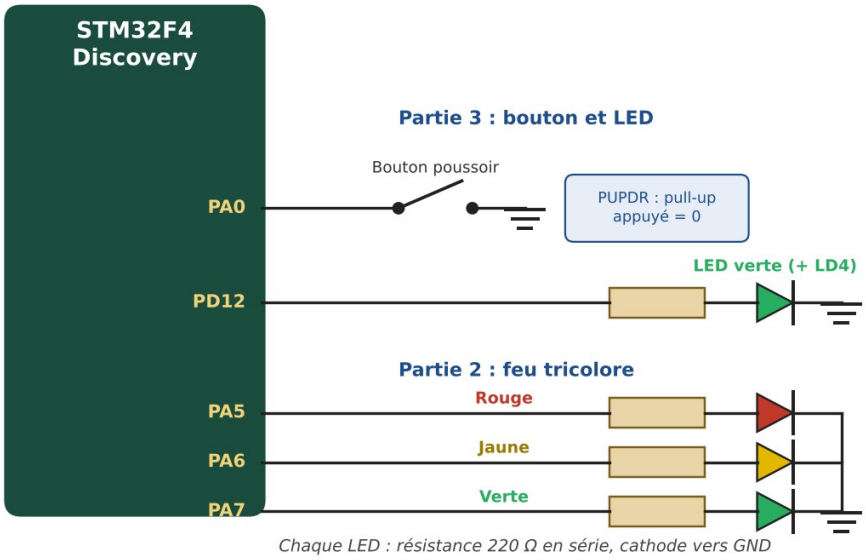


Figure 2 – *Schéma de câblage des parties 2 et 3*

Chaque LED est reliée à sa broche à travers une résistance de 220 Ω, sa cathode étant reliée à la masse. Le bouton poussoir est câblé entre PA0 et GND : grâce à la résistance de tirage interne vers 3,3 V (pull-up), la broche lit 1 au repos et 0 lorsque le bouton est appuyé.

3. Rappel : les registres GPIO utilisés

Registre	Rôle	Utilisation dans ce TP
`RCC->AHB1ENR`	Active l'horloge des ports GPIO.	Bits GPIOAEN et GPIODEN pour les ports A et D.
`MODER`	Mode de chaque broche sur 2 bits : 00 entrée, 01 sortie.	PA0, PA1 en entrée ; PD12, PA5, PA6, PA7 en sortie.
`PUPDR`	Résistance de tirage sur 2 bits : 00 aucune, 01 pull-up, 10 pull-down.	Pull-up (01) sur PA0 et PA1.

Registre	Rôle	Utilisation dans ce TP
IDR	Lecture de l'état des broches en entrée.	Test du bit 0 pour lire le bouton.
ODR	Écriture de l'état des broches en sortie.	Feu tricolore : une seule LED allumée à la fois.
BSRR	Bits 0 à 15 : mise à 1 d'une broche ; bits 16 à 31 : mise à 0.	Bit 12 : allume PD12 ; bit 28 (12 + 16) : l'éteint.

Le registre BSRR permet de modifier une seule broche en une seule écriture, sans lire le registre au préalable : écrire 1 dans le bit n met la broche n à 1, écrire 1 dans le bit n + 16 la met à 0, et les bits à 0 n'ont aucun effet. C'est plus sûr qu'une opération « lecture-modification-écriture » sur ODR.

4. Partie 1 : rappel – clignotement d'une LED sur PA5

Avant de passer aux entrées, nous avons vérifié le montage avec le programme du TP précédent : la broche PA5 est configurée en sortie et son état est inversé en boucle, ce qui fait clignoter une LED externe. La valeur de la boucle d'attente a été augmentée par rapport au TP précédent pour que le clignotement soit visible à l'œil.

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;      // horloge du port A
GPIOA->MODER &= ~(3U << (5 * 2));         // PA5 en sortie
GPIOA->MODER |=  (1U << (5 * 2));
while (1) {
    GPIOA->ODR ^= (1U << 5);              // inversion de PA5
    for (volatile int i = 0; i < 500000; i++);
}
```

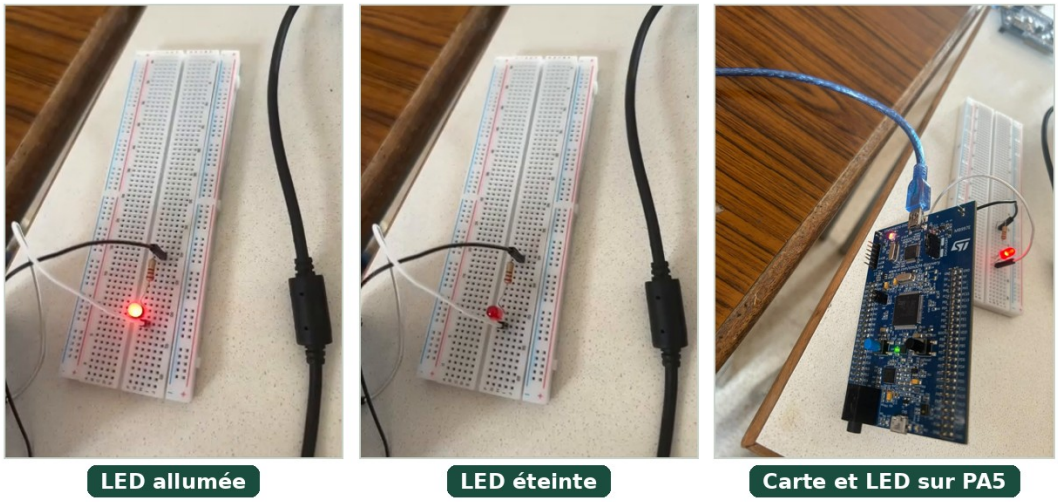


Figure 3 – *Clignotement de la LED externe sur PA5 (images extraites de la vidéo et photo du montage)*

La LED s'allume et s'éteint régulièrement : la carte, le câblage et la chaîne de compilation et de téléversement de Keil fonctionnent correctement.

5. Partie 2 : feu tricolore

5.1 Programme

```
#include "stm32f407xx.h"

void delay(volatile int t)
{
    while (t--);
}

int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

    GPIOA->MODER &= ~((3U << 10) | (3U << 12) | (3U << 14));
    GPIOA->MODER |=  ((1U << 10) | (1U << 12) | (1U << 14));

    while (1)
    {
        GPIOA->ODR = (1U << 5);    // rouge
        delay(8000000);
        GPIOA->ODR = (1U << 6);    // jaune
        delay(8000000);
        GPIOA->ODR = (1U << 7);    // vert
        delay(8000000);
        GPIOA->ODR = (1U << 6);    // jaune
        delay(8000000);
        GPIOA->ODR = (1U << 5);    // rouge
        delay(8000000);
    }
}
```

5.2 Explication du programme

Instruction	Rôle
`void delay(volatile int t) { while (t--); }`	Temporisation logicielle : décompte t jusqu'à 0. Le mot-clé volatile empêche le compilateur de supprimer la boucle.
`GPIOA->MODER &= ~((3U << 10) | (3U << 12) | (3U << 14))`	Efface en une seule instruction les bits de mode de PA5 (bits 10-11), PA6 (12-13) et PA7 (14-15).
`GPIOA->MODER |= ((1U << 10) | (1U << 12) | (1U << 14))`	Écrit 01 pour les trois broches : PA5, PA6 et PA7 en sortie.
`GPIOA->ODR = (1U << 5)`	Affectation de tout le registre : PA5 passe à 1 et toutes les autres sorties du port A à 0. Une seule LED est donc allumée.
`delay(8000000)`	Maintient l'état pendant quelques secondes.

L'utilisation de « = » au lieu de « |= » sur ODR est ici un choix pratique : chaque écriture allume une LED et éteint automatiquement les deux autres, sans instruction supplémentaire. La séquence obtenue est **rouge → jaune → vert → jaune → rouge**, puis la boucle recommence.

5.3 Durée des temporisations

Après un reset, le STM32F407 fonctionne sur son oscillateur interne HSI à 16 MHz. Chaque tour de la boucle while (t--) demande plusieurs cycles d'horloge, car la variable volatile est relue et réécrite en mémoire à chaque tour. Avec environ 6 à 8 cycles par tour, delay(8000000) dure de l'ordre de 3 à 4 secondes. Cette durée dépend du compilateur et de son niveau d'optimisation : une temporisation logicielle n'est pas précise, contrairement à un timer matériel.

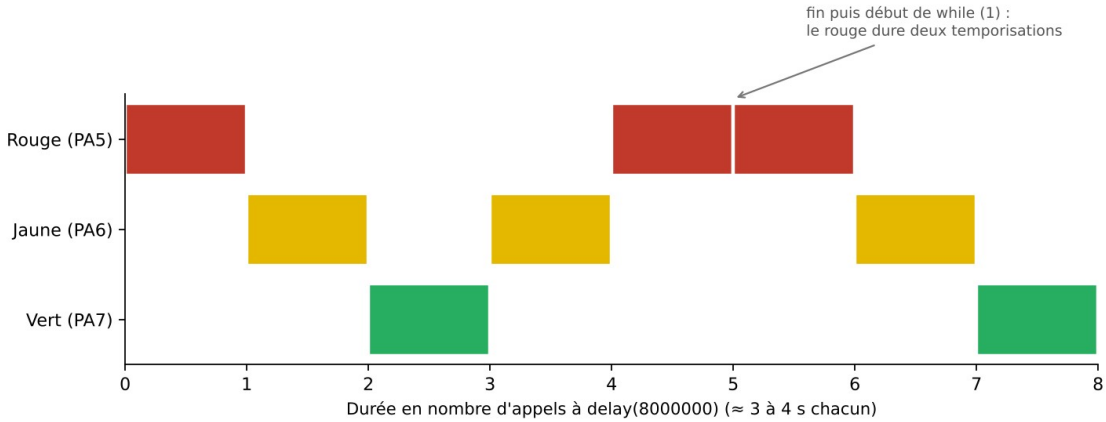


Figure 4 – *Séquence d'allumage des trois LED du feu tricolore*

Comme la boucle se termine par la LED rouge et recommence par la LED rouge, le rouge reste allumé pendant deux temporisations consécutives, soit environ deux fois plus longtemps que les autres couleurs. On pourrait supprimer la dernière écriture de la boucle pour obtenir des phases de même durée.

5.4 Résultats

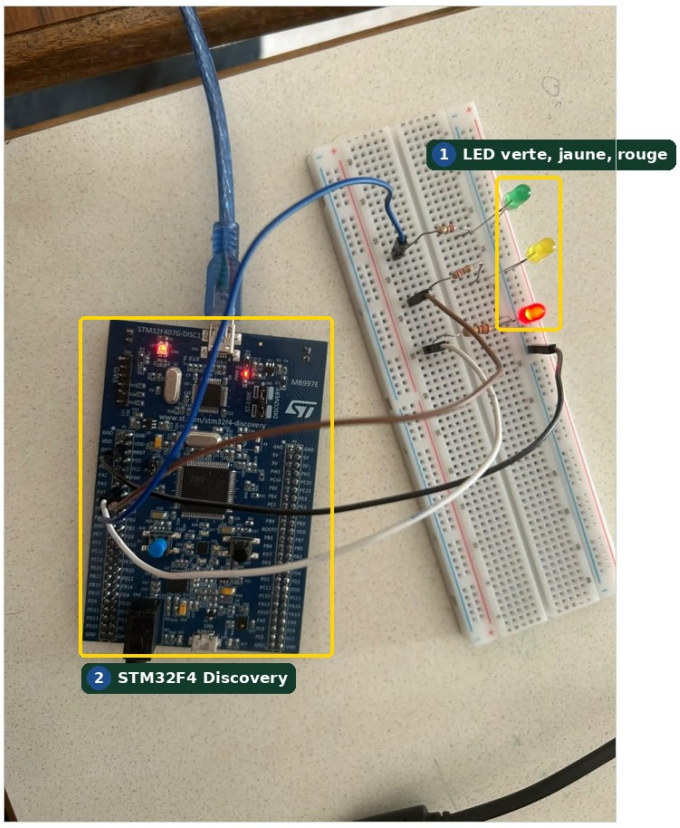


Figure 5 – *Montage du feu tricolore sur la carte Discovery*

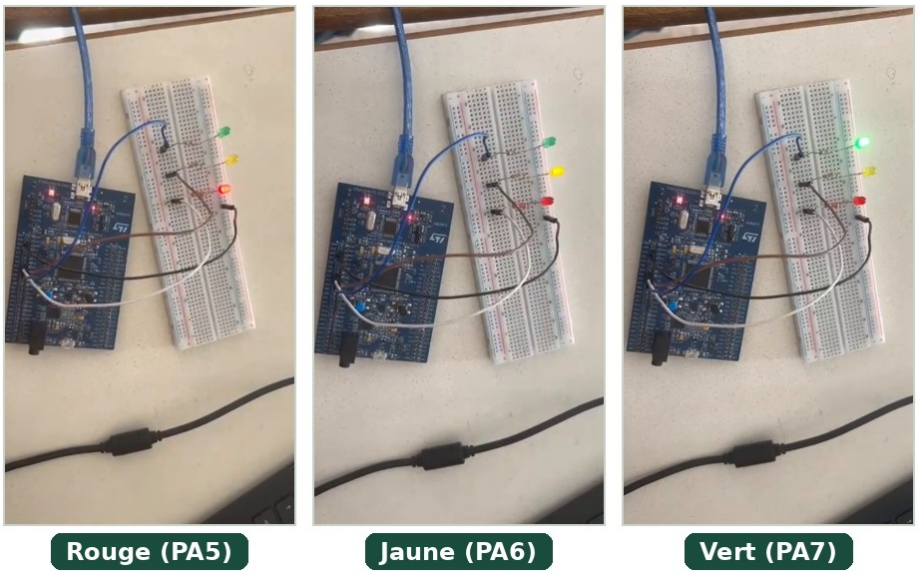


Figure 6 – *Séquence du feu tricolore extraite de la vidéo de la séance*

Les images extraites de la vidéo montrent successivement la LED rouge, puis la jaune, puis la verte allumées seules, conformément au programme.

6. Partie 3 : commande d'une LED par un bouton poussoir

6.1 Programme saisi

L'objectif est de commander la LED reliée à PD12 à l'aide d'un bouton poussoir. Comme le montre la photo du montage (figure 8), un seul bouton est câblé sur la breadboard. Le programme saisi dans Keil configure pourtant deux entrées, PA0 et PA1, comme s'il était prévu pour deux boutons (un pour allumer, un pour éteindre) :

```
#include "stm32f407xx.h"

int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIODEN;

    GPIOD->MODER &= ~(3U << (12 * 2));
    GPIOD->MODER |=  (1U << (12 * 2));

    GPIOA->MODER &= ~(3U << (0 * 2));
    GPIOA->MODER &= ~(3U << (1 * 2));

    GPIOA->PUPDR &= ~(3U << (0 * 2));
    GPIOA->PUPDR |=  (1U << (0 * 2));
    GPIOA->PUPDR &= ~(3U << (1 * 2));
    GPIOA->PUPDR |=  (1U << (1 * 2));

    while (0)
    {
        if ((GPIOA->IDR & (1U << 0)) == 0)
        {
            GPIOD->BSRR = (1U << 12);
        }
        if ((GPIOA->IDR & (1U << 1)) == 1)
        {
            GPIOD->BSRR = (1U << 28);
        }
    }
}
```

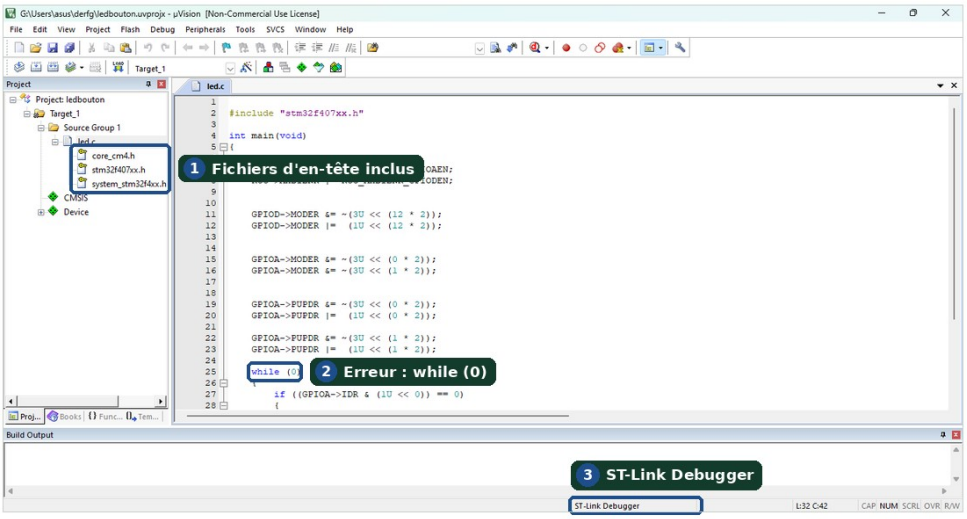


Figure 7 – *Programme saisi dans Keil µVision (projet ledbouton)*

6.2 Explication du programme

Instruction	Rôle
`RCC->AHB1ENR |= ...GPIOAEN / GPIODEN`	Active l'horloge des ports A (bouton) et D (LED).
`GPIOD->MODER ... (12 * 2)`	Met les bits 24-25 à 01 : PD12 en sortie.
`GPIOA->MODER &= ~(3U << (0 * 2)) ...`	Met les bits de PA0 et PA1 à 00 : entrées.
`GPIOA->PUPDR ... 1U << (0 * 2)`	Active la résistance de pull-up sur PA0 et PA1 (code 01).
`(GPIOA->IDR & (1U << 0)) == 0`	Isole le bit 0 de IDR : vaut 0 si le bouton relié à PA0 est appuyé.
`GPIOD->BSRR = (1U << 12)`	Met PD12 à 1 : la LED s'allume.
`GPIOD->BSRR = (1U << 28)`	Bit 28 = 12 + 16 : met PD12 à 0, la LED s'éteint.

6.3 Erreurs relevées dans le programme

L'analyse du programme saisi fait apparaître deux erreurs qui l'empêchent de fonctionner comme prévu, ainsi que deux remarques liées au montage :

Ligne	Problème	Conséquence
`while (0)`	La condition 0 est toujours fausse : le corps de la boucle n'est jamais exécuté.	Après la configuration, main() se termine : le bouton n'est jamais lu et la LED ne réagit pas.
`(IDR & (1U << 1)) == 1`	Le masque (1U << 1) donne 0 ou 2 (valeur du bit 1), jamais 1.	La condition est toujours fausse : cette partie du programme ne peut jamais éteindre la LED.
`PA1`	Aucun bouton n'est relié à PA1 sur le montage.	Grâce au pull-up, PA1 lit toujours 1 : même corrigé, ce test ne servirait à rien avec un seul bouton.

Ligne	Problème	Conséquence
`PA0`	Sur la carte Discovery, PA0 est aussi reliée au bouton utilisateur bleu B1.	Un appui sur B1 force aussi PA0 ; il faut en tenir compte lors des essais.

Pour tester un bit, il faut comparer le résultat du masque à 0 (« == 0 » ou « != 0 »), et non à 1, car le masque conserve la position du bit : `IDR & (1U << 1)` vaut 2 lorsque le bit 1 est à 1.

6.4 Programme corrigé

Avec un seul bouton, il faut que ce même bouton allume et éteigne la LED. La solution la plus simple est que la LED suive l'état du bouton : allumée tant qu'il est appuyé, éteinte dès qu'il est relâché. Seule la boucle principale doit être modifiée ; la configuration de PA1 peut être conservée ou supprimée, car elle n'est plus utilisée :

```
    while (1)                                   // boucle infinie
    {
        if ((GPIOA->IDR & (1U << 0)) == 0)      // bouton appuyé (PA0 = 0)
        {
            GPIOD->BSRR = (1U << 12);           // PD12 = 1 : LED allumée
        }
        else                                    // bouton relâché (PA0 = 1)
        {
            GPIOD->BSRR = (1U << (12 + 16));    // PD12 = 0 : LED éteinte
        }
    }
```

Avec le pull-up, un bouton appuyé donne 0 : le test utilise donc « == 0 ». La boucle while (1) lit le bouton en permanence, et chaque écriture dans BSRR ne modifie que PD12, sans toucher aux autres broches du port D.

6.5 Montage et comportement attendu

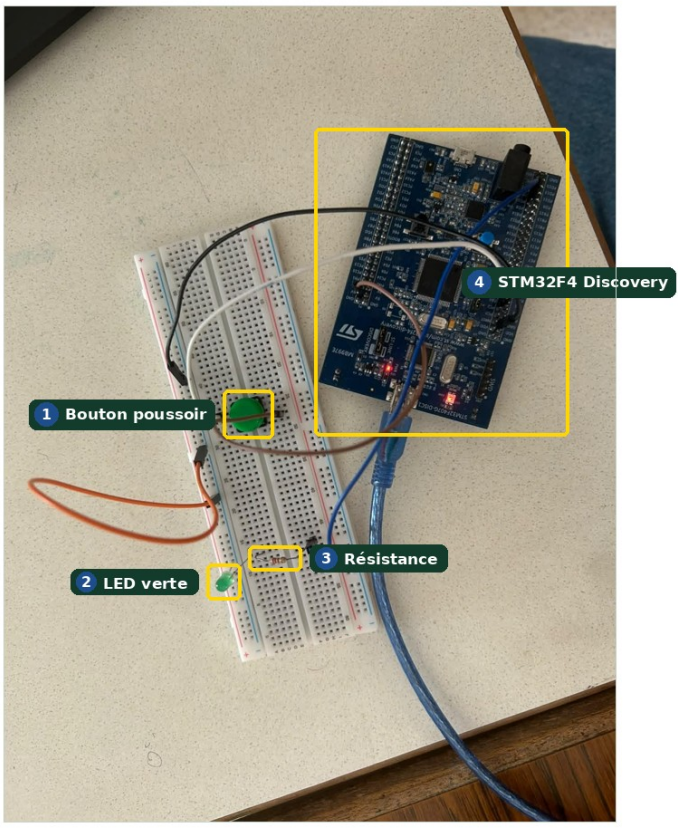


Figure 8 – *Montage de la partie 3 : bouton poussoir, LED et carte Discovery*

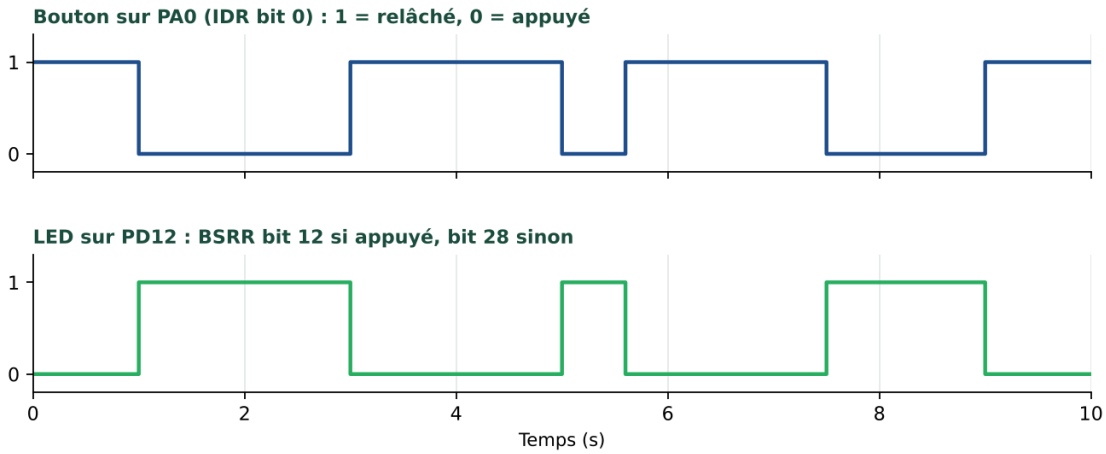


Figure 9 – *Comportement attendu du programme corrigé*

Tant que le bouton est maintenu appuyé, PA0 vaut 0 et la LED est allumée ; dès qu'il est relâché, PA0 revient à 1 grâce au pull-up et la LED s'éteint. Comme PD12 est aussi reliée à la LED verte LD4 de la carte, celle-ci s'allume en même temps que la LED externe.

7. Points d'attention

- **Horloge des ports :** sans l'activation dans RCC->AHB1ENR, les registres du port ne réagissent pas.
- **Test d'un bit :** comparer un masque à 0, jamais à 1 lorsque le bit n'est pas le bit 0.

- **Boucle principale :** un programme embarqué doit tourner en permanence, d'où while (1) et non while (0).
- **ODR et BSRR :** une affectation sur ODR modifie tout le port, alors que BSRR ne touche que les broches choisies.
- **Temporisation :** une boucle vide bloque le processeur et sa durée est approximative ; un timer matériel est préférable pour une durée précise.

8. Conclusion

Ce TP nous a permis de commander plusieurs sorties puis de lire des entrées du STM32F407 en programmant directement ses registres. Nous avons réalisé un feu tricolore en écrivant dans ODR avec une temporisation logicielle, puis configuré des entrées avec résistance de pull-up (MODER, PUPDR), lu l'état d'un bouton poussoir (IDR) et commandé une LED de façon sûre avec BSRR.

L'analyse du programme du bouton nous a appris à relire attentivement un code avant de conclure : deux détails, while (0) et une comparaison à 1, suffisaient à le rendre inopérant. Les prochains TP pourront remplacer la scrutation du bouton par des interruptions externes (EXTI) et la boucle de temporisation par un timer matériel, pour obtenir un programme plus précis et non bloquant.

Remerciements

Nous remercions M. Atef Ftirich, notre enseignant de TP, pour ses explications claires, ses conseils et sa disponibilité tout au long de cette séance. Nous remercions également l'établissement pour la mise à disposition du matériel nécessaire à la réalisation de ce travail.